Academia FitLife

Da aplicação CRUD ao processamento dentro do PostgreSQL.

Organizar.
Calcular.
Registrar.

Três finalidades diferentes para mostrar o banco de dados como parte ativa do sistema.

O problema

Uma academia precisa relacionar alunos e planos, acompanhar matrículas e registrar recebimentos. O CRUD anterior já guardava esses dados; a evolução acrescenta consultas reutilizáveis, cálculo financeiro e validação de pagamentos no banco.

A solução implementada

Recurso	Responsabilidade	Funcionalidade
View	Organizar dados relacionados	Listagem de matrículas
Function	Calcular um resultado	Total pago por aluno
Procedure	Executar um processo	Registrar pagamento

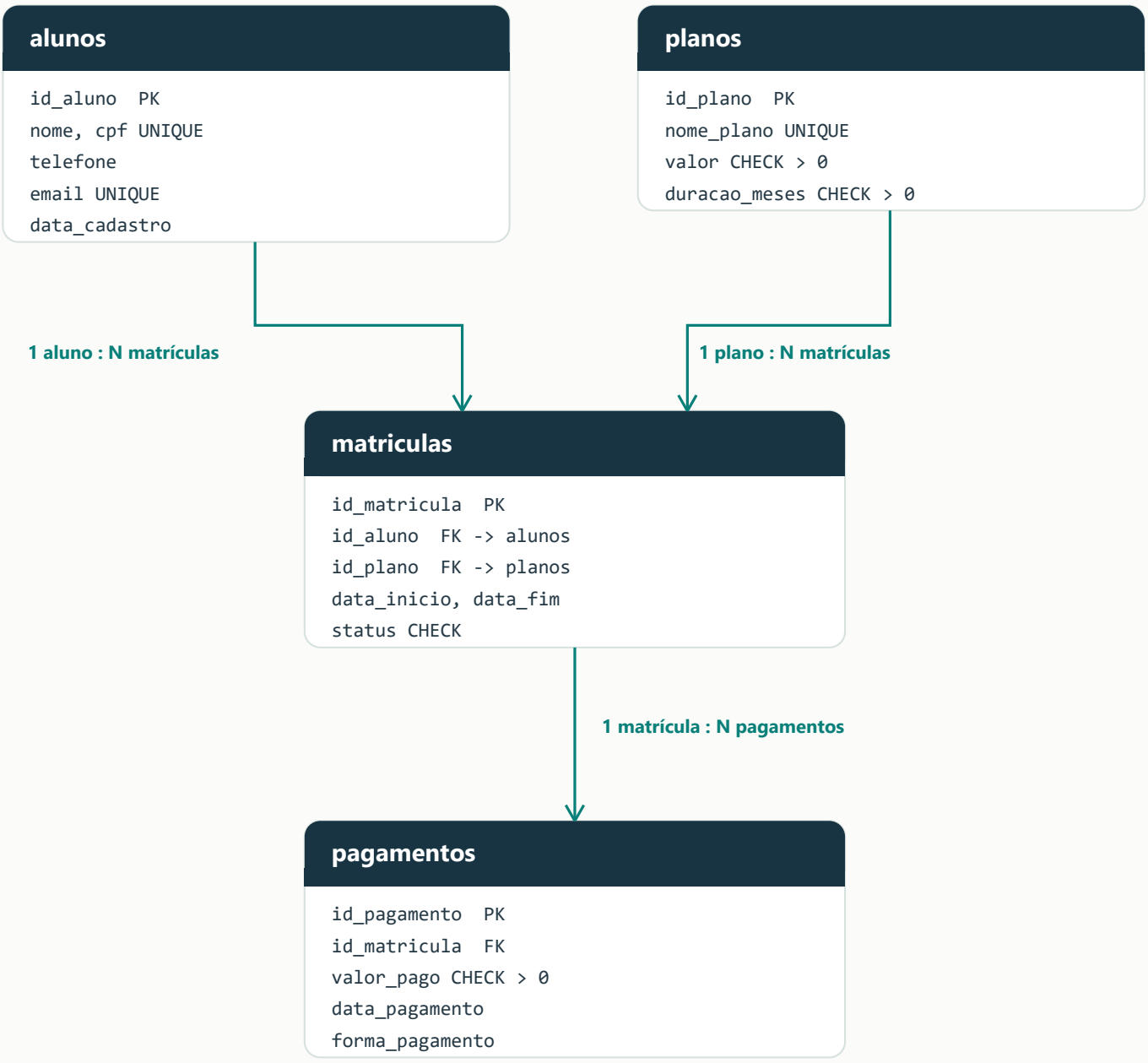
PostgreSQL 18 **Python + Tkinter** **psycopg2**

Erick Ruan Nunes Vieira
Banco de Dados - Prof. Anderson Costa

Trabalho individual. Interface com Alunos, Matrículas, Pagamentos e Relatórios. Desenvolvimento com auxílio de IA e validação manual do autor.

Cinco tabelas, vínculos claros

Chaves e restrições preservam os relacionamentos do sistema.



usuarios

```
id_usuario  PK
login UNIQUE
senha VARCHAR(100)
criado_em
```

Conta de acesso não é aluno.
usuarios atende à autenticação, sem vínculo direto com matrículas.

Exclusões: aluno e matrícula usam CASCADE para os registros dependentes. Um plano referenciado usa RESTRICT. A interface confirma as exclusões antes de executá-las.

View: matrículas consolidadas

Finalidade: reunir os dados que a interface precisa apresentar.

Banco_de_dados/ddl/view.sql

```
CREATE OR REPLACE VIEW vw_matriculas_alunos AS
SELECT
    a.nome AS aluno,
    p.nome_plano AS plano,
    m.status,
    m.id_matricula,
    m.data_inicio,
    m.data_fim
FROM matriculas AS m
INNER JOIN alunos AS a
    ON m.id_aluno = a.id_aluno
INNER JOIN planos AS p
    ON m.id_plano = p.id_plano;
```

Os dois INNER JOINs ligam cada matrícula ao seu aluno e ao plano escolhido. A View retorna nomes, situação, identificador e datas. Ela guarda a definição da consulta; os dados continuam nas tabelas.

Chamada efetiva pela aplicação

banco_matriculas.py / listar_matriculas()

```
SELECT id_matricula, aluno, plano, status,
       data_inicio, data_fim
FROM vw_matriculas_alunos
ORDER BY aluno, data_inicio DESC;
```

Aluno	Plano	Situação
Carlos Henrique Souza	Trimestral	CANCELADA
Fernanda Lima Rocha	Anual	ATIVA
João Pedro Alves	Mensal	ATIVA

Onde aparece

Aba Matrículas e relatório de matrículas ativas. O botão Atualizar consulta novamente a View. A tabela acima ilustra os dados usados durante a demonstração.

Function: total pago por aluno

Entrada: ID do aluno. Saída: valor NUMERIC calculado pelo PostgreSQL.

Banco_de_dados/ddl/function.sql

```
CREATE OR REPLACE FUNCTION fn_total_pago_aluno(p_id_aluno INTEGER)
RETURNS NUMERIC
LANGUAGE sql
AS $$
    SELECT COALESCE(SUM(pg.valor_pago), 0)
    FROM pagamentos pg
    JOIN matriculas m
        ON m.id_matricula = pg.id_matricula
    WHERE m.id_aluno = p_id_aluno;
$$;
```

O que o SQL faz

JOIN encontra as matrículas do aluno e seus pagamentos. SUM soma valor_pago. COALESCE substitui NULL por zero quando a soma não encontra valores. O resultado é o total histórico pago, inclusive de matrículas canceladas ou vencidas.

banco_relatorios.py / total_pago_aluno()

```
return consultar(
    "SELECT fn_total_pago_aluno(%s)",
    (id_aluno,)
)[0][0]
```

Antes	Pagamento adicional	Depois
R$ 89,90	R$ 10,00	R$ 99,90

Evidência de integração

Na demonstração manual, o aluno 3 passou de R$ 89,90 para R$ 99,90 após registrar R$ 10,00. O Python apenas formata a moeda; o cálculo continua na Function.

Limite da consulta

Zero representa ausência de pagamentos encontrados. A Function atual não diferencia um aluno sem pagamentos de um ID inexistente e não calcula saldo devedor.

Procedure: pagamento validado

Uma operação do sistema executada dentro de uma transação.

Banco_de_dados/ddl/procedure.sql

```
CREATE OR REPLACE PROCEDURE pr_registrar_pagamento(
    p_matricula INTEGER,
    p_valor NUMERIC,
    p_forma VARCHAR
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_status VARCHAR;
BEGIN
    SELECT status INTO v_status
    FROM matriculas
    WHERE id_matricula = p_matricula
    FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Matrícula não encontrada.';
    END IF;
    IF v_status IS DISTINCT FROM 'ATIVA' THEN
        RAISE EXCEPTION 'A matrícula precisa estar ATIVA.';
    END IF;
    IF p_valor IS NULL OR p_valor <= 0
       OR p_valor::TEXT IN ('NaN', 'Infinity', '-Infinity') THEN
        RAISE EXCEPTION 'Informe um valor positivo e válido.';
    END IF;
    IF p_forma IS NULL
       OR p_forma NOT IN ('PIX', 'CARTAO', 'DINHEIRO') THEN
        RAISE EXCEPTION 'Forma de pagamento inválida.';
    END IF;
    INSERT INTO pagamentos (
        id_matricula, valor_pago, forma_pagamento
    )
    VALUES (p_matricula, p_valor, p_forma);
END;
$$;
```

Parâmetros: p_matricula INTEGER, p_valor NUMERIC e p_forma VARCHAR. FOR UPDATE mantém a linha da matrícula bloqueada até o fim da transação. RAISE EXCEPTION interrompe uma operação inválida.

banco_pagamentos.py / registrar_pagamento()

```
cur.execute(
    "CALL pr_registrar_pagamento(%s, %s, %s)",
    (id_matricula, valor, forma)
)
```

A aba Pagamentos chama a Procedure. banco.transacao() executa commit no sucesso e rollback no erro, fechando cursor e conexão. A data vem do padrão CURRENT_DATE da tabela.

Do clique ao resultado

O banco participa de três funcionalidades reais da interface.

Tela	Python	PostgreSQL
Matrículas	banco_matriculas.py	SELECT da View
Relatórios / total pago	banco_relatorios.py	SELECT da Function
Pagamentos / registrar	banco_pagamentos.py	CALL da Procedure

Transação e consistência

banco.py centraliza consultas e gravações. A criação de matrícula calcula a data final no PostgreSQL com a duração do plano. As operações de criar/ativar matrícula bloqueiam a linha do aluno antes de conferir outra ATIVA. Essa regra está no fluxo da aplicação, sem índice exclusivo no esquema atual.

O que foi conferido

O autor confirmou manualmente no banco real: cadastro/login; cadastro, edição e exclusão de aluno; criação, alteração e exclusão de matrícula; registro e exclusão de pagamento; consultas dos relatórios. O aumento e a redução do total pago permitiram acompanhar as mudanças no banco.

Base desta apresentação: leitura dos scripts e módulos locais e resultados relatados na demonstração. Testes simulados apoiaram o desenvolvimento, sem equivaler a uma suíte automatizada publicada. Não se afirma validação em produção.

Limites e próximas evoluções

A Procedure atual verifica status ATIVA, sem conferir vencimento. O status não muda sozinho com o tempo. As exclusões em cascata removem histórico; cancelamento preserva os pagamentos. Perfis de acesso, auditoria, cobrança recorrente e saldo devedor ficam para uma evolução futura.

> **Aprendizado central**
>
> Modelar vínculos, aplicar restrições e decidir onde processar dados: organizar com View, calcular com Function e executar um processo com Procedure. A interface torna esses recursos observáveis.

REPOSITÓRIO DO PROJETO

https://github.com/erickruannv/academia-bd

Disciplina: Banco de Dados | Prof. Anderson Costa
Autor: Erick Ruan Nunes Vieira | Desenvolvimento com auxílio de IA.